

MANTIS Extended — Post-Paper Extensions to a Banking Multi-Agent Security Testbed

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—MANTIS (Modular Agent Network Testbed for Instrumentation and Security) is a configuration-driven security and observability testbed built directly on a real banking multi-agent system spanning front-, mid-, and back-office workflows. The first paper describing MANTIS scoped its contribution deliberately: a hook bus exposing five fixed control points, a compact attack and failure plugin library, a seven-dimension automated evaluator, and a benchmarking suite – with a named, explicit list of extensions deferred to future work rather than attempted under that paper’s schedule. This paper implements four of those deferred extensions and reports live, measured evidence for each, without modifying the first paper’s own artifact; the fifth, a Zero Trust Backplane integration, is being pursued as a separate, dedicated effort and is intentionally out of scope here (§ I). Additional exporters: all four of the coding plan’s named backends – Jaeger, Grafana (Tempo), Phoenix, and Langfuse – each enabled by a one-line configuration change, that fail silently rather than aborting a run when no collector is reachable; the Langfuse adapter, whose ingestion endpoint is HTTP-only and always requires Basic auth even self-hosted, was confirmed reaching the real hosted Langfuse endpoint and receiving a genuine unauthorized response with no credentials supplied. Security mechanism plugins: a concrete guardrail, evaluated through the identical plugin interface as an attack, that we show live blocking a real attack – a tool-parameter-mutation attack inflating a transfer’s amount from \$125.50 to \$5,000 was denied before it reached the banking backend, with the denial recorded as a distinct `POLICY_EVENT` rather than misclassified as an attack. Additional banking workloads: a new front-office dispute-filing and -tracking process, added as a third route alongside the existing transaction-review and customer-service workflows rather than a new prompt into an existing one – and a debugging finding in its own right: an initial version silently produced zero tool calls across three live trials because its scenario prompt omitted a required identifier every other scenario in the codebase states explicitly, invisible until we actually inspected the emitted trace rather than a clean exit code. Minimal UI: a schema-driven experiment editor and trace viewer whose mutating actions shell out to the identical CLI subprocess a terminal user would run, verified by confirming a deliberately invalid configuration is rejected with the real CLI’s own error message rather than a UI-side check. The project’s test suite, exercised against real data rather than mocks throughout, stands at 139 passing tests as of this revision.

Index Terms—multi-agent systems, adversarial security, large language models, banking, observability, extensibility

Identify applicable funding agency here. If none, delete this.

I. Introduction

The first MANTIS paper [1] scoped its contribution narrowly on purpose: a hook bus exposing five fixed control points around an otherwise-unmodified banking multi-agent system, a compact library of attack and failure plugins registered through one shared interface, a seven-dimension automated evaluator, and a benchmarking suite isolating instrumentation overhead from LLM sampling noise. That paper’s own coding plan named five extensions explicitly deferred rather than attempted under its schedule: additional observability exporters beyond OpenTelemetry and MLflow; security-mechanism plugins (guardrails, policy checks, rate limits) evaluated through the same framework as attacks; a Zero Trust Backplane consuming the testbed’s application inventory and control-point metadata to generate or enforce protection; additional banking workloads beyond the ones the first paper’s evaluation exercised; and a minimal, schema-driven UI over the existing command-line interface.

This paper implements four of the five. The fifth, the Zero Trust Backplane, was explored to the same live-verified standard as the other four – a default-deny-by-domain policy generator and enforcement plugin, an exhaustive check of every (agent, tool) pair a live inventory implies, and repeated live trials confirming zero false-positive denials on legitimate traffic – against a real, independently-developed Zero Trust Backplane project for this same banking testbed [2]. That work is being carried forward as its own dedicated integration effort with that project rather than folded into this paper’s own repository and evidence, and is out of scope for the remainder of this paper.

Each of the four extensions covered here is built to the same standard the first paper set for itself: a real, live-verified capability rather than a description of one, checked against the actual emitted trace rather than a clean process exit, with the evidence – not just the claim – checked into the repository. We report that standard being met unevenly on purpose: the dispute-filing workload initially failed to do anything meaningful on first attempt, for reasons a single successful-looking run

would not have surfaced, and we report that failure and its root cause alongside the working result, consistent with the first paper’s own central methodological finding that verifying a security-relevant capability means inspecting what actually happened, not assuming it from a lack of an error.

None of the four extensions required a change to `mantis.core`, `mantis.hooks`, or the banking workflow modules – the same architectural claim the first paper made for its own attack and failure plugins now extends to defensive plugins and telemetry adapters as well.

II. Extension 1: Additional Observability Exporters

The first paper’s observability pipeline supported JSONL traces, MLflow run tracking, and a bare OpenTelemetry TracerProvider with no span exporter of its own attached – creatable spans that were never actually shipped anywhere. We implemented all four of the coding plan’s named additional backends: Jaeger, Grafana (Tempo), Phoenix, and Langfuse.

`mantis.observability.jaeger_exporter.setup_jaeger_otel()` attaches a real `OTLPSpanExporter` (gRPC) to the shared TracerProvider, registered in `exporter_registry` as “jaeger” alongside the existing three. Enabling it is a configuration change – `observability.export: [jsonl, jaeger]` – not a code change to the banking workflow, the hook bus, or the observability plugin. Because a run’s success must never depend on telemetry actually being delivered, we verified the failure path directly: with no collector listening at the configured endpoint, the SDK’s `BatchSpanProcessor` retries and logs a warning rather than raising, confirmed both by a direct unit test asserting no exception propagates and by observing the expected `StatusCodes.UNAVAILABLE` warnings during every subsequent offline test run in this development environment, where no Jaeger collector was running.

Grafana Tempo and Phoenix follow the identical adapter shape as Jaeger – both accept OTLP natively over gRPC, so `grafana_exporter.py` and `phoenix_exporter.py` are the same thin wrapper around `OTLPSpanExporter`, differing only in their default endpoint and an optional cloud-auth header/API key for each backend’s managed offering. Langfuse genuinely differs: its OTLP ingestion endpoint (`/api/public/otel`) is HTTP/protobuf only, never gRPC, and always requires HTTP Basic auth – a public/secret key pair – even against a self-hosted instance, so `langfuse_exporter.py` is built on `opentelemetry-exporter-otlp-proto-http` rather than the `.grpc` package the other three share. All three are registered in `exporter_registry` exactly like Jaeger, enabled the same config-only way, and verified with the same fail-silent-with-no-collector test pattern. The Langfuse adapter received a stronger live check than a local no-collector test can offer: run with its default endpoint and no credentials, it reached the real `cloud.langfuse.com` OTLP endpoint over the network and received a genuine HTTP 401 response, independently

confirming the transport and endpoint wiring are correct end-to-end against the real service, not only against an absent local collector.

III. Extension 2: Security Mechanism Plugins

The first paper shipped `PolicyPlugin` as an interface only, matching the exact shape of `ExperimentPlugin` so a future defensive plugin would not need to masquerade as an attack plugin, without a concrete implementation. We built `AmountLimitGuardrailPlugin`: a guardrail denying a tool call whose declared amount field exceeds a configured limit, through the identical `apply(ctx) -> HookResult` contract every attack plugin already uses.

The demonstration pairs this guardrail with the first paper’s own tool-parameter-mutation attack rather than an undefended baseline, because the interesting property is not that the guardrail can deny a call in isolation but that it evaluates a call an attack has already mutated. Policy plugins are registered onto the hook bus after any configured attack (mirroring the attack-before-observability ordering the first paper established, for the identical reason: a plugin registered later in a dispatch sees the effect of everything registered before it). Run live against a real model with a \$2,000 limit: `execute_transfer` was called with a legitimate \$125.50 transfer; the attack mutated it to \$5,000 destined for an account named `HACKER-9999`; the guardrail denied the mutated call before it reached the banking backend; and the agent correctly reported the block back to the customer as a payment-limit restriction rather than a generic failure.

This also required a small but consequential change to the observability layer’s own event classification. The first paper’s `ObservabilityPlugin` classified every non-trivial plugin action as `ATTACK_INJECTED` except a named set of failure-plugin names, which it classified as `ANOMALY`. A guardrail’s `DENY` is a defense working as intended, not an attack; classifying it as `ATTACK_INJECTED` would have made every future evaluator built on that event type unable to tell a caught attack apart from a successful one. We added a parallel `POLICY_EVENT` classification, keyed the same way the existing failure-plugin split already was. The live run’s trace confirms both classifications appear correctly and distinctly in the same run: one `ATTACK_INJECTED` record for the mutation, one `POLICY_EVENT` record for the denial.

IV. Extension 3: Additional Banking Workloads

The coding plan’s fourth extension calls for “more banking processes... or workflow patterns,” which we read as a requirement for a genuinely new business process, not a new prompt into an existing one. We added dispute filing and status lookup as a front-office capability: a new disputes table in the same SQLite repository `submit_manual_review` already uses, a new tool module, and `dispute_resolution_agent` registered as

a third route under `front_office_router`, alongside the existing transaction-review and customer-service-chatbot routes – distinct from the chatbot’s pre-existing FAQ-only “how do I dispute a transaction” answer, which remains purely informational and never files a real case.

The first attempt at this extension produced a run that exited cleanly and called no tool at all, across three consecutive live trials against a real model. The cause was not a defect in the tool or the agent’s routing – both worked correctly, confirmed by the real transfer to `dispute_resolution_agent` appearing in every trace – but in the scenario prompt itself, which never stated a customer identifier, unlike every other scenario in this codebase. `file_dispute(customer_id, transaction_id, reason)` requires one as its first argument; without it stated in the prompt the model apparently could not commit to calling the tool at all, silently, with no error the process’s own exit code would reveal. Adding the customer identifier to the prompt – not changing the tool’s signature – produced, on the next live run, a real case filed with a generated identifier, correctly persisted, and a correct customer-facing confirmation message referencing that identifier. We report this not as an aside but because it is the same class of finding the first paper’s own central claim predicts: a clean exit and a correctly-routed trace both looked identical whether or not the actual business action happened, and only inspecting the tool-call events specifically distinguished the two.

V. Extension 5: Minimal UI

The coding plan’s final named extension is deliberately constrained: “a schema-driven editor and trace viewer that invokes the same CLI/API; no separate business logic.” We built a small FastAPI application whose two mutating actions – validate and run – shell out to the exact mantis console-script entrypoint a terminal user would invoke, as a real subprocess, rather than a second, UI-specific implementation of configuration validation or experiment execution. Every read-only endpoint – the configuration schema, the domain/workflow/scenario/plugin registries, the live inventory, and a run’s own trace file – calls the identical underlying objects the CLI itself reads from, so the editor’s form fields are populated from live data rather than a hardcoded list that could drift from the deployed system.

We verified the “no separate business logic” property directly rather than assuming it from the implementation’s structure: submitting a configuration with a deliberately invalid domain through the UI’s validate endpoint returns the real CLI’s own “Unknown domain” error text, produced by the same subprocess a terminal invocation of `mantis -validate` would produce, not a UI-side check reproducing that logic. The application was also confirmed running as a real server, not only through an in-process test client: launched via `mantis -ui`, it correctly reported the current live inventory (32 agents, 22 tools) and the

TABLE I
Post-Paper Extensions: verification evidence.

Extension	Tests	Live evidence
Jaeger exporter	4	Real OTLP span processor attached; confirmed silent-safe with no collector listening.
Grafana/Phoenix/Langfuse	9	Same OTLP adapter pattern for the first two; Langfuse (HTTP-only, Basic auth) reached the real hosted endpoint and got a genuine 401 with no credentials.
Guardrail plugin	6	Live run: \$125.50 transfer mutated to \$5,000 by a real attack, denied by the guardrail before reaching the backend; both events correctly classified.
Dispute workload	7	Live run: real case filed and persisted after correcting a scenario-prompt defect found only by inspecting the trace.
Minimal UI	10	Real invalid config rejected with the real CLI’s own error text; real running server confirmed via HTTP, not only a test client.

full set of existing run artifacts (27 runs) over real HTTP requests.

VI. Evaluation Summary

Table I summarizes each extension’s verification evidence. All four were checked against real data or a real live run rather than a mock, consistent with the first paper’s own testing philosophy; none required a change to `mantis.core`, `mantis.hooks`, or the banking workflow modules. The offline suite – covering all four extensions, including the three additional exporters added in this revision – stands at 139 passing tests, still executing in well under a minute with no live LLM key required: 4 for the Jaeger exporter and 9 for Grafana/Phoenix/Langfuse together, 6 for the guardrail (including one exercising the real hook bus with both an attack and a guardrail plugin registered, confirming the guardrail sees the attack’s already-mutated payload), 7 for the dispute workflow, and 10 for the UI, on top of the base suite the first paper’s own work established.

VII. Conclusion

We implemented four of the five extensions the first MANTIS paper explicitly deferred, to the same evidentiary standard that paper set for itself: live-verified behavior, checked against the actual emitted trace, with the evidence checked into the repository rather than described in prose. One surfaced a real defect on first implementation – a scenario-design gap that silently produced zero tool calls across three live trials – invisible from a clean process exit, found only because that extension’s own verification pass insisted on checking real behavior rather than accepting that a run had completed. Revisiting the exporters extension specifically, to replace a single-backend anecdote with broader coverage, found that the remaining three named exporter backends were still unbuilt; all three are now live-verified to the same standard as the first. None of the four required a change to the first paper’s core interfaces. The fifth extension, the Zero Trust Backplane,

was built and repeated-trial-verified to this same standard – including an exhaustive check of every (agent, tool) pair a live inventory implies, and a real event-misclassification defect found and fixed along the way – but is being carried forward as a dedicated integration effort with a real, independently-developed Zero Trust Backplane project for this same testbed [2] rather than reported as part of this paper’s own repository and evidence. Future work includes repeated-trial efficacy statistics for the guardrail’s own denial path, and continuing that separate Zero Trust integration effort toward a production integration between this testbed’s inventory and a real signed decision pipeline.

Acknowledgment

References

- [1] [Authors], “MANTIS – Modular Agent Network Testbed for Instrumentation and Security in Multi-Agent Banking Systems,” 2026. (Companion paper; this work’s Section II onward assumes its architecture and terminology.)
- [2] smartsystemslab-uf, “Citi P3 Zero Trust Backplane,” github.com/smartsystemslab-uf/Citi_P3_Zero_Trust_Backplane, 2026.